Performance benchmarking of cryptographic mechanisms

Group: Filipe Constantino Reis 202307182

Maria Luiza de Almeida Pereira 202304440

Erling Heimstad Willassen 202512252

Class: Pl 4

Introduction

In cryptography there exists multiple algorithms that are widely used for different purposes such as confidentiality, integrity, and secure key exchange. Among these algorithms we will analyse three of them:

AES (Advanced Encryption Standard): Symmetric-key Algorithm that is often used for bulk data encryption.

RSA (Rivest–Shamir–Adleman): Asymmetric-key Algorithm that is used for secure key exchange.

SHA-256: Hash function made to ensure data integrity.

This report presents a structured analysis of each algorithm with the Goal to benchmark, compare and find out the best scenario for the use of all 3 algorithms.

Experimental Setup

Hardware and Software Specifications

CPU: Intel I5-13400F

RAM: 16G

Operating system: Windows

Programming language: python version 3.14.3

Libraries: Time, Cryptograph, Statistics, Pandas, matplotlib, binascii, pathilb

Methodology

Disable CPU turbo boost/power-saving mode;

Avoid having background processes running;

Warm-up 10 times for more consistent results;

Each experiment is done 100 times;

Input data

Random files generated with 8B, 64B, 512 B, 4 KB, 32 KB, 256 KB, 2 MB of size

Files were generated with the “os.urandom(size)” python function, that gives a random character's string of a specified size.

Code Implementation

AES Implementation

For AES we use a key of 32 bytes, IV of 16 bytes and AES in Counter mode.

AES benchmarking variables

Throughput;

Mean Encryption/Decryption time;

Standard deviation;

Median;

Encryption Function

```
CTR_256_encrypt(message, size, state)
```

To Encrypt the message, we use pythons Cryptograph library.

Cipher creates a cipher of a specific Algorithm and mode, in this case AES in Counter mode, the encryptor checks if both variables are valid and the encrypted_msg uses the encryption on a plain text.

```
cipher = Cipher(algorithms.AES(key), modes.CTR(iv))
encryptor = cipher.encryptor()
encrypted_msg = encryptor.update((message)) + encryptor.finalize()
```

Decryption Function

```
CTR_256_decrypt(encrypted_message, size, state)
```

To decrypt the message, we also pythons Cryptograph library.

```
cipher = Cipher(algorithms.AES(key), modes.CTR(iv))
```

```
decryptor = cipher.decryptor()
decryptor.update(encrypted_message) + decryptor.finalize()
```

RSA Implementation

For RSA we use a key of 2042 bits (the minimum recommended for RSA).

We use a modified RSA that generates a random value r with 32 bytes, encrypts r with RSA, uses r to generate a keystream using sha256 and finally XOR's the keystream with the plaintext.

This modification is used because to avoid encrypting large data with RSA.

RSA benchmarking variables

Mean Encryption/Decryption time;

Throughput;

Standard deviation;

Encryption Function

```
rsa_cryptograph(mensagem, size, state)
```

This part of the code generates the r and encrypts it with RSA.

```
r = os.urandom(32)
r_int = int.from_bytes(r, byteorder='big')
r_encrypt_int = encrypt(r_int, e, n)
r_encrypt = r_encrypt_int.to_bytes(256, byteorder='big')
```

The For cycle generates the keystream and OXR's it with the plain text in blocks

```
for i in range(0, size, tamanho_bloco):
        r_atual = counter.to_bytes(4, byteorder='big') + r
        r_sha = hashlib.sha256(r_atual).digest() #H(i, r)
        block_cipher = bytes(a ^ b for a, b in zip(mensagemDividida, r_sha))
        cipher_text.extend(block_cipher)
encrypted_msg = bytes(cipher_text)
```

Decryption Function

```
rsa_decryptograph(cipher, size, state)
```

This code piece grabs the r and decrypts it

```
    r_cipher_bytearray = cipher[:256] # cipher r
    cipher_text = cipher[256:] #cipher text without r
    r_cipher_int = int.from_bytes(r_cipher_bytearray, byteorder='big')
    r_int = decrypt(r_cipher_int, d, n)
```

```
    r = r_int.to_bytes(32, byteorder='big')
```

The For cycle calculates the hash of r and XOR's with the cypher text

```
 for i in range(0, len(cipher_text), tamanho_bloco):
     r_atual = counter.to_bytes(4, byteorder='big') + r
     r_sha = hashlib.sha256(r_atual).digest() #H(i, r)
     block_decipher = bytes(a ^ b for a, b in zip(mensagemDividida, r_sha))
     plain_text.extend(block_decipher)
decrypted_msg = bytes(plain_text)
```

SHA-256 implementation

SHA-256 benchmarking variables

Throughput;

Time per hash;

Decryption Function

```
sha_cryptograph(mensagem, size, state)
```

we use pythons "hashlib" library to create the hash.

```
encrypted_msg = hashlib.sha256(mensagem).digest()
return bytes(encrypted_msg)
```

Functions used in All codes

Benchmark function

```
timer(message, size, time_list, state, function)
```

Function to calculate the time it takes to complete one "function" and puts the results in a list.

It runs the function 10 times to warm up and then converts the time to microseconds.

```
for _ in range(10): # warmup function :)
    function(message, size, state)
time1 = timeit.default_timer() # timer starts here
function(message, size, state)
time2 = timeit.default_timer() # timer ends here
total_time = (time2 - time1)* 1_000_000 #time is in microsseconds
time_list.append(total_time) # appends time use to list
```

Stats function

```
calculate_stats(times_us)
```

Function that given a list of times calculates the mean, standard deviation, and median using the python library Statistics.

```
mean_us = statistics.mean(times_us) # creating mean of times in list
std_us = statistics.stdev(times_us) # creating standard deviation of times in list
```

```
median_us = statistics.median(times_us) # creating median of time in list
```

Test function

```
enc_dec_test()
```

Function that tests the encryption and decryption and puts the results in a dictionary.

It runs the function repetitions times to reduce computer instabilities and to give more consistent results.

```
for _ in range(repetitions):
   timer(message, size, time_list_enc, False, CTR_256_encrypt)
enc_message = CTR_256_encrypt(message, size, True)
for _ in range(repetitions):
  timer(enc_message, None, time_list_dec, None, CTR_256_decrypt)
enc_mean, enc_std, enc_median = calculate_stats(time_list_enc)
dec_mean, dec_std, dec_median = calculate_stats(time_list_dec)
enc_throughput = (size / enc_mean) / (1024 * 1024)
dec_throughput = (size / dec_mean) / (1024 * 1024)
```

Results

AES

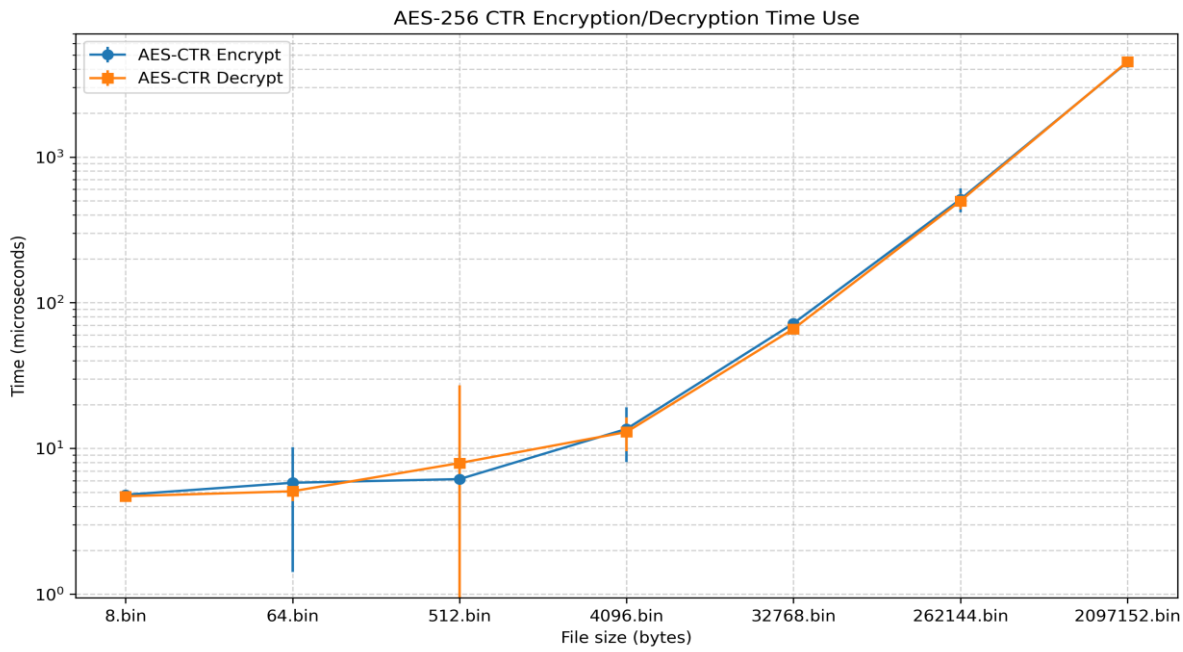

Table of results

File Size (bytes)	Enc Mean (µs)	Enc Median (µs)	Enc Std	Enc Throughput (MB/s)	Dec Mean (µs)	Dec Median (µs)	Dec Std	Dec Throughput (MB/s)

			Dev (µs)				Dev (µs)	
8	4.81	4.80	0.16	0.002	4.70	4.70	0.12	0.002
64	5.81	5.10	4.40	0.012	5.08	5.00	0.75	0.012
512	6.16	5.70	3.34	0.086	7.94	5.60	19.20	0.087
4,096	13.60	12.60	5.55	0.310	12.96	12.50	3.38	0.312
32,768	72.14	72.10	4.04	0.433	66.17	64.00	5.62	0.488
262,144	512.47	484.65	94.19	0.516	499.06	480.80	61.90	0.520
2,097,152	4470.59	4438.85	132.87	0.451	4509.99	4451.80	165.11	0.449

RSA

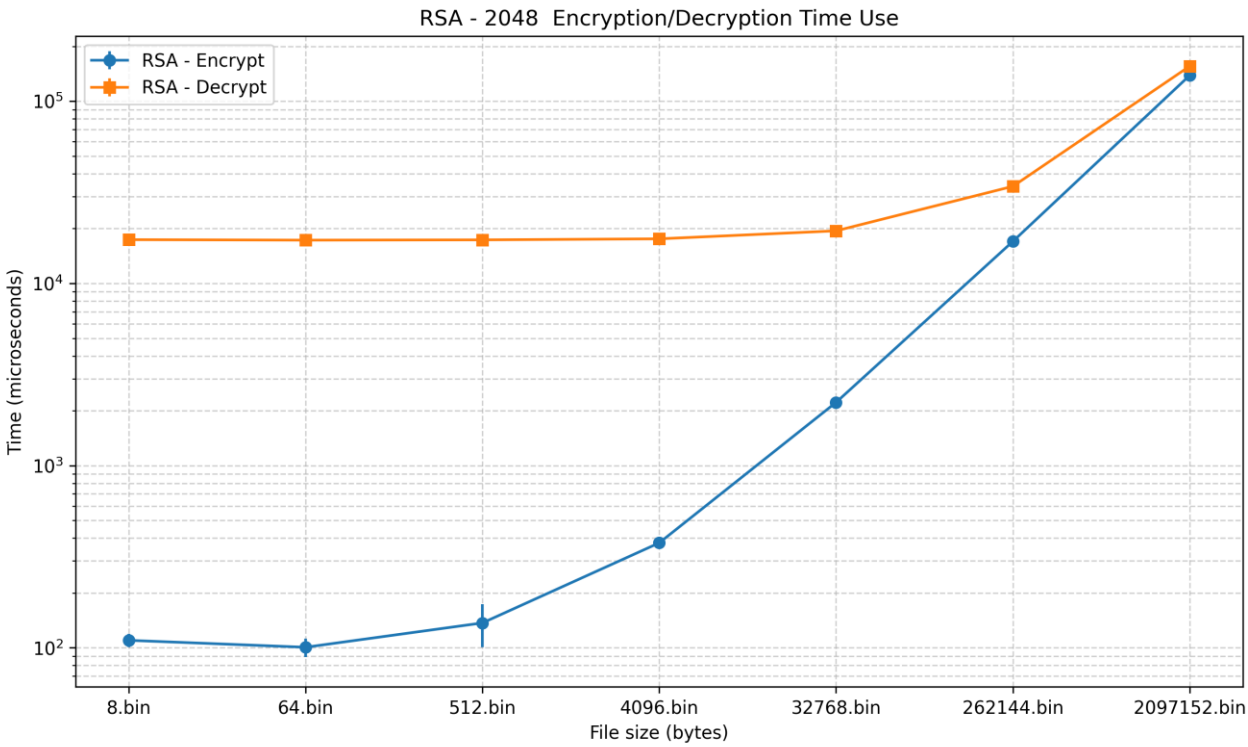

Table of results

File Size (bytes)	Enc Mean (µs)	Enc Median (µs)	Enc Std Dev (µs)	Enc Through put (MB/s)	Dec Mean (µs)	Dec Median (µs)	Dec Std Dev (µs)	Dec Through put (MB/s)
8	109.82	107.10	9.20	0.069	17,386.65	17,268.20	656.26	0.0004

64	100.61	98.30	11.94	0.607	17,283.35	17,265.05	113.23	0.0035
512	136.81	128.95	36.30	3.569	17,343.01	17,320.20	105.77	0.028
4,096	376.42	365.70	25.43	10.38	17,571.68	17,528.25	146.61	0.222
32,768	2,223.10	2,205.05	69.93	14.06	19,439.52	19,379.20	262.00	1.608
262,144	17,016.66	16,947.20	290.66	14.69	34,120.26	34,074.10	164.82	7.327
2,097,152	138,791.34	138,008.35	3,204.30	14.41	155,907.66	155,935.85	780.88	12.83

28.26 keys generated per second.

SHA-256

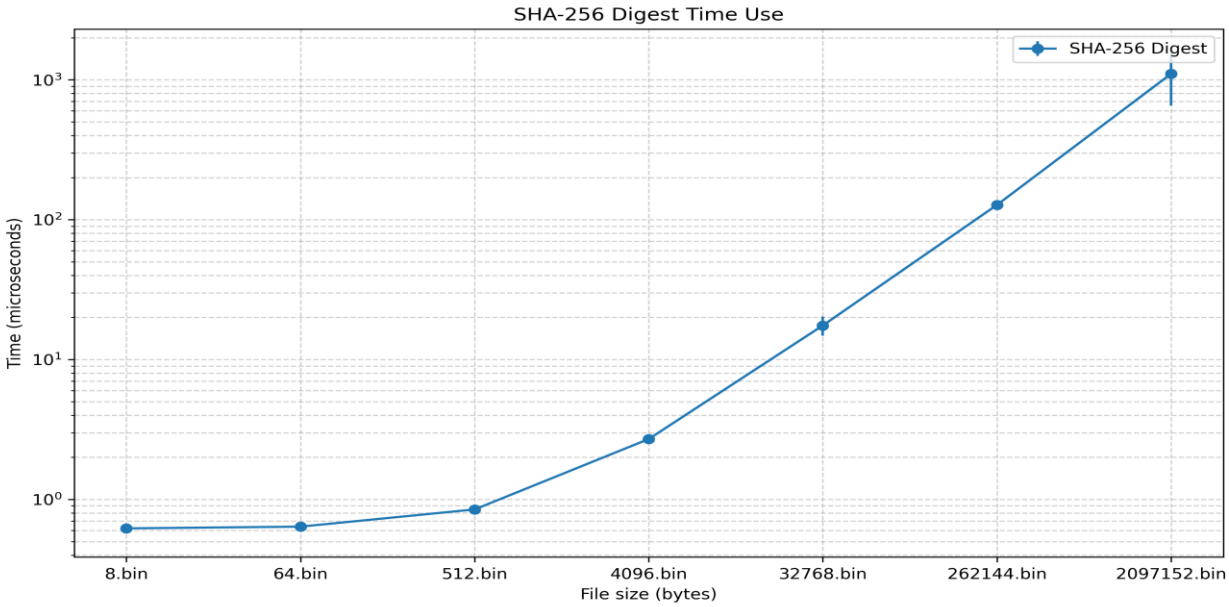

Table of results

File Size (bytes)	Mean Time (µs)	Median (µs)	Std Dev (µs)	Throughput (MB/s)
8	0.62	0.60	0.05	12.29
64	0.64	0.60	0.05	95.52
512	0.85	0.80	0.05	575.80

4,096	2.70	2.70	0.04	1445.69
32,768	17.52	17.10	2.73	1783.57
262,144	127.39	131.80	7.31	1962.49
2,097,152	1102.32	1056.45	449.24	1814.35

Analysis and Observations

AES Encryption VS RSA Encryption

AES and RSA serve two distinct purposes in cryptography, AES is a symmetric key algorithm while RSA is asymmetric, AES is better for larger data sets, but RSA is more secure because it uses a public and private key.

As we can see in the graph and table AES is significantly faster to encrypt than RSA, AES scales linearly with file size while RSA scales with a steep curve.

AES also has a much faster throughput in the bigger files that RSA meaning it is better for bigger files.

This shows that AES is a better algorithm to use on bulk files, but in general it would be better to use a combination of both RSA to generate keys and AES for the bulk file.

AES Encryption VS SHA-256 digest generation

AES and SHA-256 have different goals, AES serves to generate confidentiality while SHA gives us integrity.

In the graph/tables we can see that SHA is faster to encrypt that ASE, that is because while AES performs multiple mathematical operations while SHA only does a few operations.

Both AES and SHA scale linearly with file size although AES is more unstable in smaller files than SHA.

RSA Encryption VS Decryption

RSA encryption uses a public key to generate a cypher text that can be decrypted using a private key.

In the graph we can see that encryption is a lot faster that decryption for smaller files but for bigger ones the times start converging, that is because the operation for

decryption is only performed once while the hashing and XOR's are performed multiple times in the encryption.

Conclusion

This benchmarking project proved the differences between the algorithm, showing the expected results of AES being better for large scale cryptograph, our modified RSA minimizing the issues that RSA has with larger files but it still being an issue and SHA being very fast which is good for integrity.

References

https://www.youtube.com/watch?v=D_PfV_IcUdA

https://www.youtube.com/watch?v=IJwWJD5N_DY

https://www.youtube.com/watch?v=9m8enHN13mw

Criptografia e Segurança de Redes. 6º edição

https://hrodriches.medium.com/encripta%C3%A7%C3%A3o-rsa-com-python-3-6d7620f167c2

https://pt.linkedin.com/pulse/how-generate-rsa-key-pairs-python-simple-guide-secure-ribeiro-cissp-hjcye?tl=pt

https://www.w3schools.com/python/ref_func_pow.asp

https://www.bearssl.org/speed.html

https://docs.nordicsemi.com/bundle/ncs-2.4.1/page/nrf/samples/crypto/aes_ctr/README.html

https://infosecwriteups.com/breaking-down-sha-256-algorithm-2ce61d86f7a3

https://zk-plus.github.io/tutorials/basics/hashing-algorithms-benchmarks